



UNIVERSITY
OF HULL

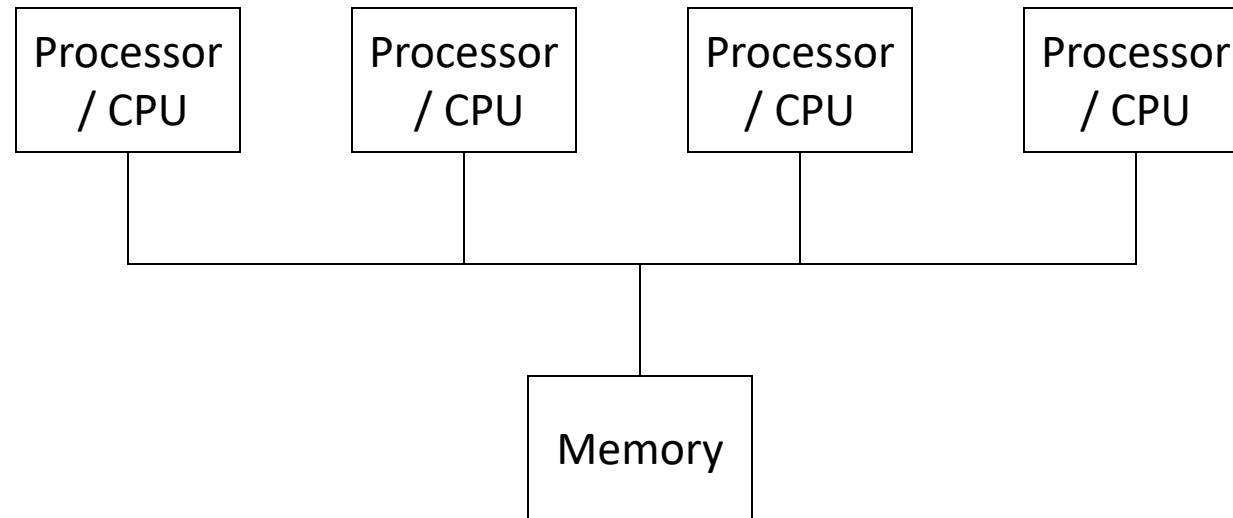
OpenMP

Overview, parallel for, performance

OpenMP

- Interface for parallelizing code
 - Portable
 - Scalable
 - High-level
 - Flexible
 - Standardized
 - Performance-oriented
- Assumes shared-memory model

Shared memory model



Background to OpenMP

- First public release 1997, stable 2002 (version 2)
- Created primarily for research and supercomputing communities
- Some relevant game compilers
 - Intel C++ 8.1
 - Microsoft Visual Studio 2005+
 - GCC (see GOMP)
- www.openmp.org

OpenMP for C/C++

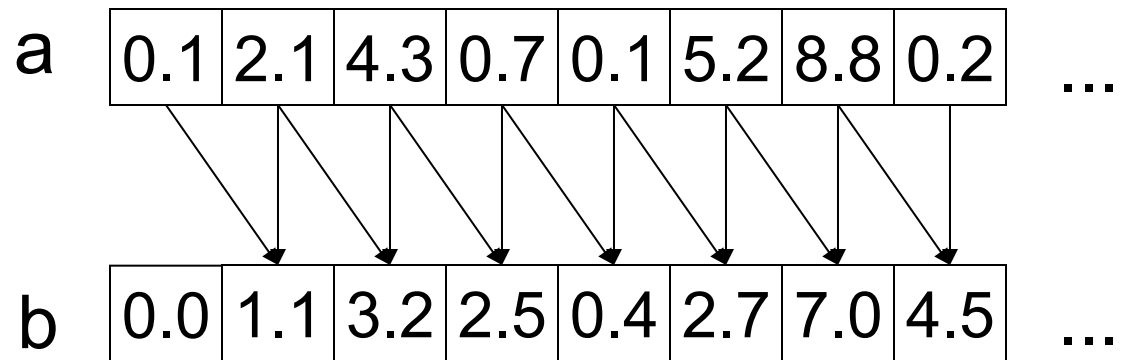
- Directives activate OpenMP
 - `#pragma omp <directive> [clauses]`
 - Define parallelizable sections
 - Ignored if compiler doesn't support OMP
- APIs
 - Configuration (e.g. # threads)
 - Synchronization primitives
- Remember to enable in compiler options

Where to start

- Include omp.h to use any structs/APIs
 - `#include <omp.h>`
- Inform your build tools
 - Set compiler flag; e.g. `/openmp`
 - Link with library; e.g. `vcomp[d].lib`
 - Or tick openmp support in project options
- Verify compiler support
 - `#ifdef _OPENMP`
 - `cout << "OpenMP enabled" << endl;`
 - `#endif`
- Add `#pragma omp`

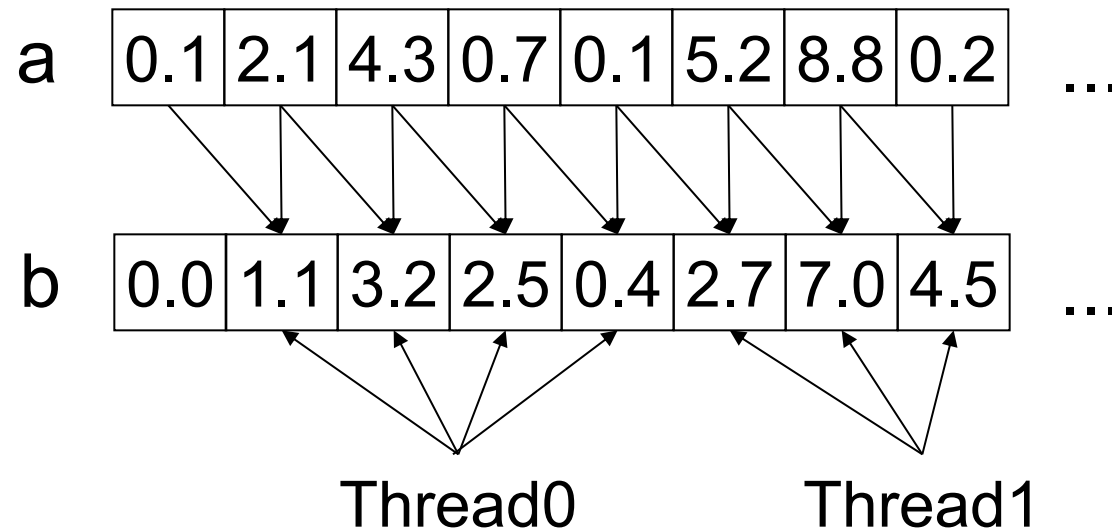
For loop

- double a[n], b[n];
- for(int i=1; i < n; ++i)
- b[i] = (a[i] + a[i-1]) / 2.0;



Parallel for loop

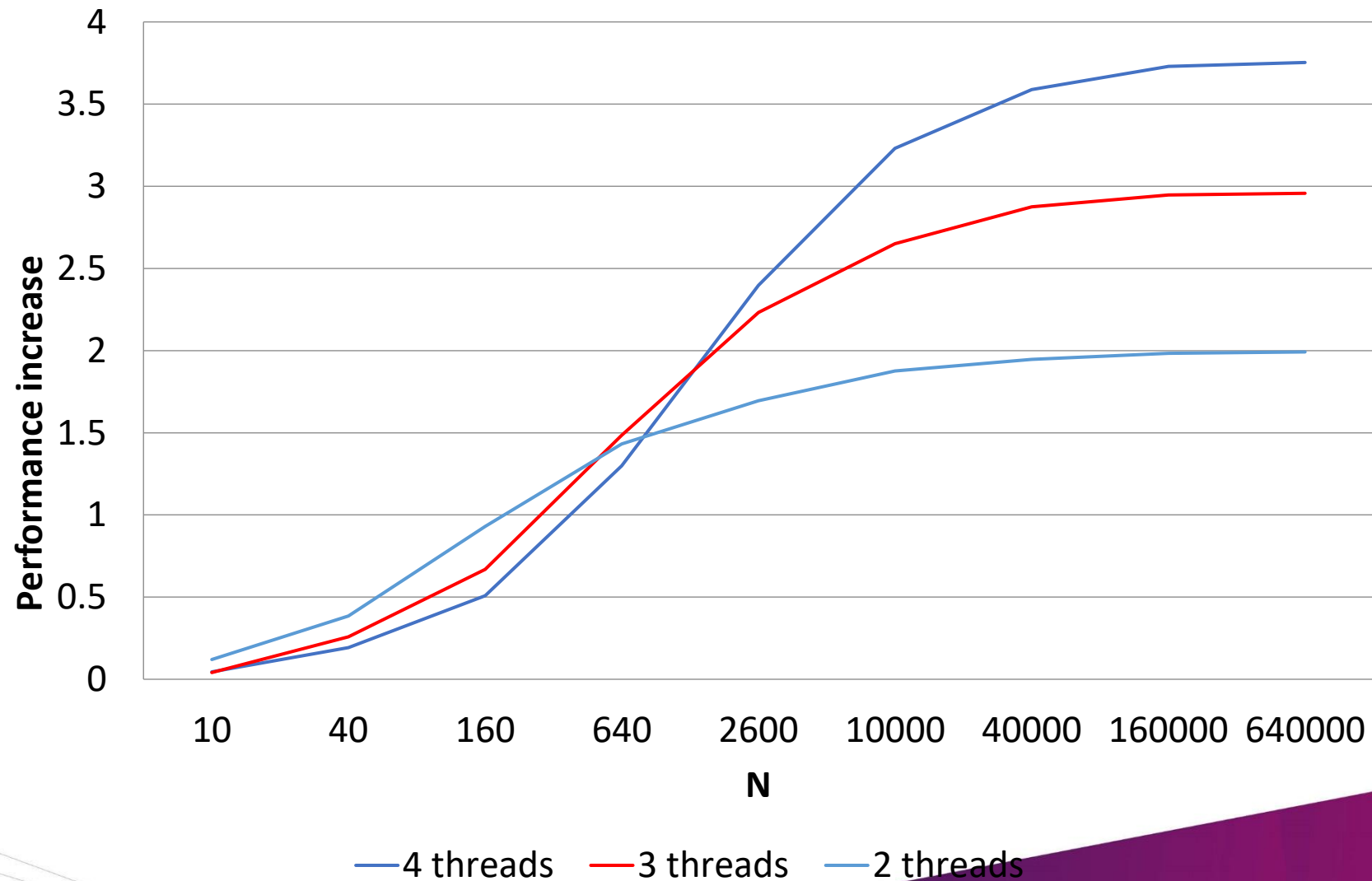
- `#pragma omp parallel for`
- `for( i=1; i < n; ++i )`
- `b[i] = (a[i] + a[i-1]) / 2.0;`



Performance of Example

- `#pragma omp parallel for`
- `for( i=1; i < n; ++i )`
- `b[i] = (a[i] + a[i-1]) / 2.0;`

Size of N



Performance Measurements

- Increase in performance is approximately
 - $1/(S + (1-S)/n)$
- Where
 - S is percentage of time spent in sequential portion of the code
 - N is number of cores
- Example
 - S=0.5, n=2 gives speed up of 1.33
 - S=0.5, n=8 gives speed up of 1.77

Compare with Windows Threads

1. `DWORD ThreadFn( VOID* pData ) { // Primary function`
2. `for( int i = pData->Start; i < pData->Stop; ++i )`
3. `b[i] = (a[i] + a[i-1]) / 2.0;`
4. `return 0; }`
5. `...`
6. `for( int i=0; i < n; ++i ) // Create thread team`
7. `hTeam[i] = CreateThread( 0, 0, ThreadFn, pDataN, 0, 0 );`
8. `// Wait for completion`
9. `WaitForMultipleObjects( n, hTeam, TRUE, INFINITE );`
10. `for( int i=0; i < n; ++i ) // Clean up`
11. `CloseHandle( hTeam[i] );`

Performance of Native Threads

- Same performance as OpenMP
 - But 10X more code to write
 - Not cross platform
 - Doesn't scale

What's the Catch?

- Performance gains depend on N and the work in the loop
- Usage restricted
 - Simple For loops
 - Parallel code sections
- Operations must be order-independent

Amdahl's rule

- Increase in performance is
 - $1/(S + (1-S)/n + H(n))$,
- Where
 - S is percentage of time spent in sequential portion of the code
 - N is number of cores
 - H(n) is overhead, relative to number of cores

Test designs

- Care is needed when performing timed tests
- Consider
 - $N=100,000,000$
 - `for( i=1; i < n; ++i )`
 - `b[i] = (a[i] + a[i-1]) / 2.0;`
- Results over 8 successive runs (in ms)

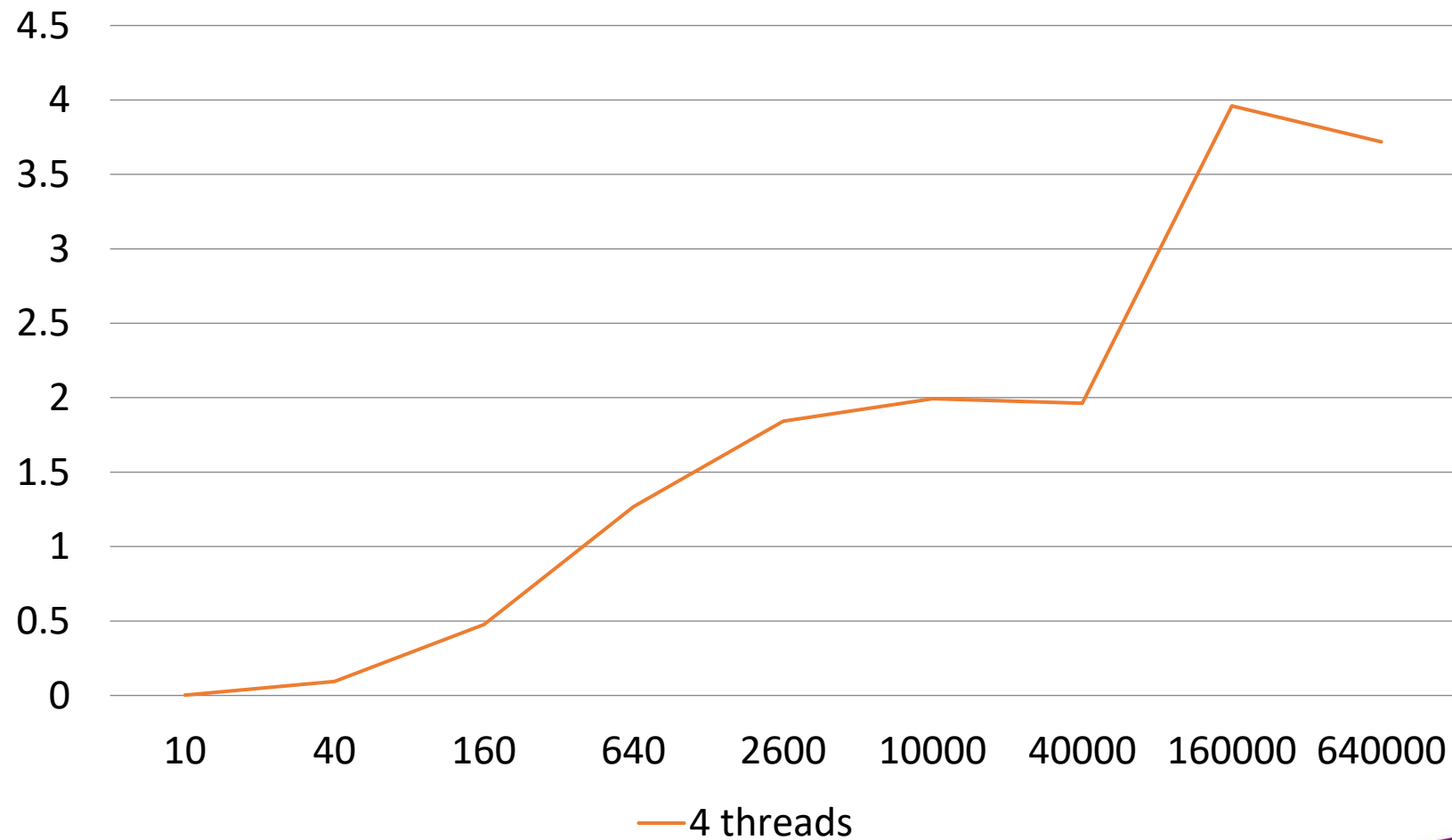
1 thread 1st run	1 thread 2nd run
4055	1627
55138	1065
8740	1041
1010	1044
1027	978
906	1058
897	1052
895	984

Chrono



- `#include <iostream>`
- `#include <chrono>`
-
- `int main() {`
- `auto start = std::chrono::high_resolution_clock::now();`
- `...`
- `auto end = std::chrono::high_resolution_clock::now();`
- `auto elapsed_seconds = end-start;`
- `...`
- `}`

Poor iteration count



Parallel for loop limitations

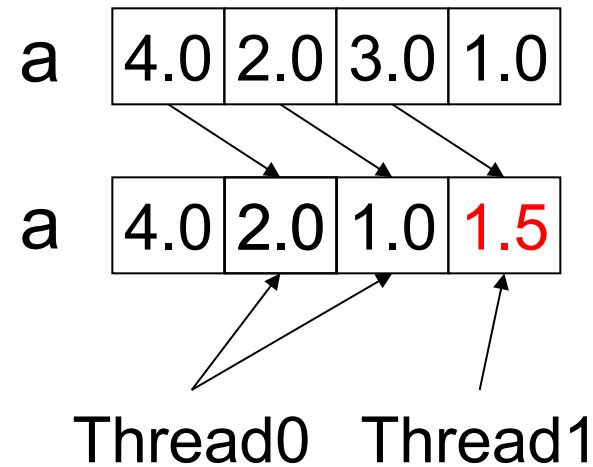
- For (i=initial; i<limit; i++)
 - i must be an integer
 - initial = invariant (constant within the scope of the loop)
 - limit = invariant (constant within the scope of the loop)
- Logical comparison only: <, <=, >, >=
- Increment: ++, --, +=, -=, +/- invariant
- No breaks allowed (break, return, exit, goto)
- Continue is allowed

STL

- Parallelizing an STL loop
 - `#pragma omp parallel for`
 - `for( itr i = v.begin(); i != v.end(); ++i )`
 - `// ...`
- Does not fit the required form

Independent Calculations

- Consider:
 - `#pragma omp parallel for`
 - `for( i=1; i < n; ++i )`
 - `a[i] = a[i-1] * 0.5;`



Correct answer

a	4.0	2.0	1.0	0.5
---	-----	-----	-----	-----

Programmers responsibility

- Verify performance gain
- Loops must be order-independent
 - Compiler cannot usually help you
 - Validate results
 - Assertions or other checks
- Be able to toggle OpenMP
 - Set thread teams to max 1
 - `#ifdef USE_OPENMP`
 - `#pragma omp parallel for`
 - `#endif`

Thread context

- When a new thread is spawned, it is provided with its own private stack.
- The private stack contains private variables, accessible only by that thread
- The new thread can also access the
 - Heap
 - Main threads stack, excluding any private variables

Variables

- Shared variables can be accessed across different threads
- By default all variables are shared, unless:
 - They are declared local to the parallel block
 - They are explicitly defined as private
 - The loop variable is implicitly private
- Private variable can be accessed only by current thread
- Private variables are undefined at the start of the parallel block
- Example
 - `#pragma omp parallel for private (j)`
 - `for( i=1; i < n; ++i )`
 - `for( j=1; j < n; ++j )`
 - `a[i] = b[j] * 0.5;`

OpenMP variables

```
1. int k, n = 10;  
2. #pragma omp parallel for  
3. for( int i=1; i < n; ++i )  
4.     for( int j=1; j < n; ++j ) {  
5.         k = b[j] * 0.5;  
6.         a[i] = k;  
7.     }
```

OpenMP variables

- `int k=1, n = 10;`
- `#pragma omp parallel for lastprivate(k)`
- `for( int i=1; i < n; ++i )`
- `for( int j=1; j < n; ++j ) {`
- `k = b[j];`
- `a[i] = k * 0.5;`
- `}`
- `cout << k;`

OpenMP variables

- `int k=1, n = 10;`
- `#pragma omp parallel for private(k)`
- `for( int i=1; i < n; ++i )`
- `for( int j=1; j < n; ++j ) {`
- `k = b[j];`
- `a[i] = k * 0.5;`
- `}`
- `cout << k;`

Variables

- Firstprivate
 - Enables initialisation of private variable before entering the parallel blocks
- Lastprivate
 - Enables access to the private variable once the parallel blocks have terminated

Configuration APIs

- `#include <omp.h>`
- `// examples`
- `int n = omp_get_num_threads();`
- `omp_set_num_threads( 4 );`
- `int c = omp_get_num_procs();`
- `omp_set_dynamic( 16 );`

OpenMP implementation

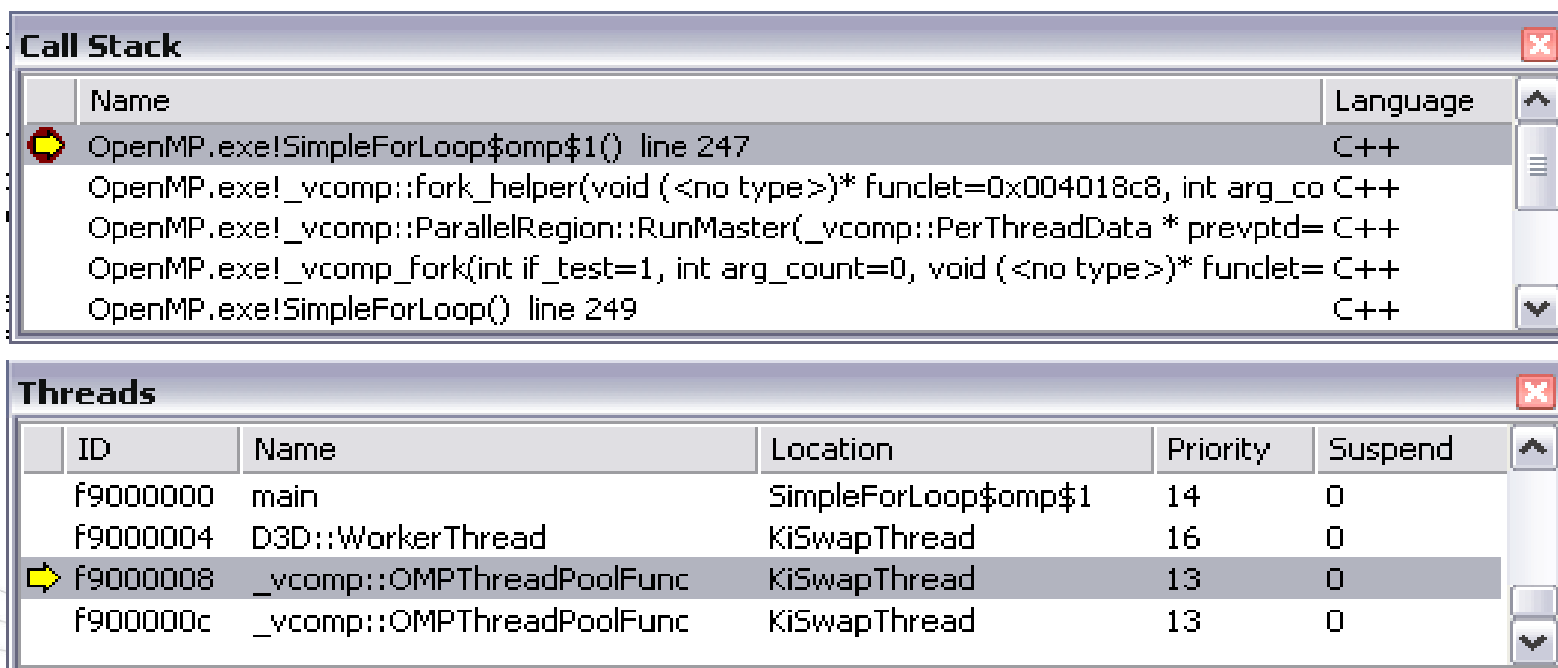
- Compiler checks OpenMP conformance
- Injects code for `#pragma omp` blocks
- Debugging runtime checks for deadlocks
- Thread team created at app startup
- Per-thread data allocated when `#pragma` entered
- Work divided into coherent chunks

Debugging

- Thread debugging is hard
- OpenMP → black box
 - Presents even more challenges
- Much depends on compiler/IDE
- Visual Studio allows breakpoints in parallel sections
- Use `omp_get_thread_num()` to get thread ID

VS Debugging Example

- `#pragma omp parallel for`
- `for( i=1; i < n; ++i )`
- `b[i] = (a[i] + a[i-1]) / 2.0; // breakpoint`



The screenshot shows two windows from the Visual Studio debugger. The 'Call Stack' window displays the current call stack with the top frame selected, which is `OpenMP.exe!SimpleForLoop$omp$1() line 247` in C++. Below it are several frames related to OpenMP's internal functions. The 'Threads' window shows a list of threads. The thread with ID `f9000008` and name `_ycomp::OMPThreadPoolFunc` is selected, showing its location as `KISwapThread` and priority as 13.

Call Stack					
	Name	Language			
1	OpenMP.exe!SimpleForLoop\$omp\$1() line 247	C++			
2	OpenMP.exe!_ycomp::fork_helper(void (<no type>)* funclet=0x004018c8, int arg_co	C++			
3	OpenMP.exe!_ycomp::ParallelRegion::RunMaster(_ycomp::PerThreadData * prevptd=	C++			
4	OpenMP.exe!_ycomp_fork(int if_test=1, int arg_count=0, void (<no type>)* funclet=	C++			
5	OpenMP.exe!SimpleForLoop() line 249	C++			

Threads					
ID	Name	Location	Priority	Suspend	
f9000000	main	SimpleForLoop\$omp\$1	14	0	
f9000004	D3D::WorkerThread	KISwapThread	16	0	
f9000008	_ycomp::OMPThreadPoolFunc	KISwapThread	13	0	
f900000c	_ycomp::OMPThreadPoolFunc	KISwapThread	13	0	

Common Problems

- Parallelizing STL loops
- The early-out problem
- Scheduling unpredictable work
- Parallelizing pointer-chasing loops (see later)

STL Loops

- For STL vector/deque
 - `#pragma omp parallel for`
 - `for( size_type i = 0; i < v.size(); ++i )`
 - `// use v[i]`

Early out

- The problem
 - `#pragma omp parallel for`
 - `for( int i = 0; i < n; ++i )`
 - `if( FindPath( i ) ) break; // compile error`
- Solutions
 - May be faster to process all paths anyway
 - Process in multiple chunks

Scheduling unpredictable work

- The problem
 - `#pragma omp parallel for`
 - `for( int i = 0; i < n; ++i )`
 - `f( i );` // f takes variable time
- Solution
 - `#pragma omp parallel for schedule(dynamic)`
 - `for( int i = 0; i < n; ++i )`
 - `f( i );` // f takes variable time

When to choose OpenMP

- Platform is multi-core with shared memory
- Profiling shows a need: 1 core is fully utilized
- Inner loops where:
 - N or loop work is significantly large
 - Processing is order-independent
 - Loops follow OpenMP canonical form
- Cross-platform important
- Last-minute optimizations

Game Applications

- Particle systems
- Skinning
- Collision detection
- Simulations (e.g. pathfinding)
- Transforms (e.g. vertex transforms)
- Signal processing
- Procedural synthesis (e.g. clouds, trees)
- Fractals

Best Practices

- Understand when it's useful
- Measure performance
- Validate results in debug mode
- Be able to turn it off

OpenMP 4.0+

- Support for code to be offloaded to the GPU
- Uses the target directive
 - `#pragma omp target \`
 - `map(to: N, a[0:N], b[0:N]), \`
 - `map(from: c[0:N])`
 - `{`
 - `int i;`
 - `#pragma omp parallel for`
 - `for (i=0; i<N; i++)`
 - `c[i] = a[i] + b[i];`
 - `}`

Auto Parallelization in VS

- Auto-Parallelization can be enabled on the VS compiler (/Qpar)
- The compiler analyses loops and determines if they can be parallelised using similar techniques to those found in OpenMP
- A hint can also be added to the code to help the compiler pinpoint the appropriate sections
 - `#pragma loop(hint_parallel(8))`
 - `for (int i=0; i<limit; ++i)`
 - `A[i] = B[i] * C[i];`

Auto-Vectorization in VS

- Auto-Vectorization is now enabled by default on the VS compilers
- The compiler analyses loops and determines if they can be vectorised using SSE instruction sets
- Auto-Vectorization can be disabled from within the code
 - `#pragma loop(no_vector)`
 - `for (int i = 0; i < 1000; ++i)`
 - `A[i] = B[i] + C[i];`
- <http://blogs.msdn.com/b/nativeconcurrency/archive/2012/04/12/auto-vectorizer-in-visual-studio-11-overview.aspx>

Threading in Python

Overview

Creating and deleting threads

Python

- High level scripting language
- First public release, 1991
- Supports object oriented and functional programming paradigms
- Used extensively in many application domains
 - UI, database, client/server, scientific, games
- Written in C, thus extremely portable
- Comprehensive set of standardised libraries
 - Including networking and threading support
- www.python.org

Why Python?

- Written in C, and uses standard threading and networking libraries.
- Effective for rapid prototyping
- Used extensively in the games industry

Creating Python Threads

- Steps in creating a thread
 - Define a function, threadMain, that is to be called when the thread is first created
 - Request the creation of a thread
 - Request the starting of the thread
 - Execution begins
 - Execution continues until threadMain completes or the thread is terminated
- threadMain is also known as threadFunction or threadProcedure

Basic threading

```
1. def threadMain(i):
2.     sleep(i)
3.     print ('thread',i,'sleeping for',i,'secs')

4. def main():
5.     threads = []
6.     threadIds = range(4)

7.     for i in threadIds:
8.         t = threading.Thread(target = threadMain, args = [i])
9.         threads.append(t)
10.
11.    for t in threads:
12.        t.start()

13.    for t in threads:
14.        t.join()
```

thread 0 sleeping for 0 secs
thread 1 sleeping for 1 secs
thread 2 sleeping for 2 secs
thread 3 sleeping for 3 secs

Basic threading – errors

```
1. def threadMain(i):
2.     sleep(0.5)
3.     print ('thread',i,'sleeping for',i,'secs')

4. def main():
5.     threads = []
6.     threadIds = range(4)

7.     for i in threadIds:
8.         t = threading.Thread(target = threadMain, args = [i])
9.         threads.append(t)

10.    for t in threads:
11.        t.start()

12.    for t in threads:
13.        t.join()
```

threadthreadthreadthread
1230 sleeping forsleeping
forsleeping forsleeping for
1230 secssecssecssecs

Thread class functionality

- start()
 - Begin execution of the thread
 - run()
 - Method to be overridden by threadMain
 - join()
 - Suspends until the thread terminates
 - getName()
 - setName()
 - isAlive()
-
- Note: Refers to class Threading.Thread